Classificazione di Domande TREC: Confronto tra BERT e GPT-2 al variare della dimensione del training set

Denise Cilia Eleonora Giuffrida

Natural Language Processing - A.A. 2025/2026

Sommario

In questo progetto sono state studiate e confrontate due architetture Transformer, BERT (encoder) e GPT-2 (decoder), sul task di classificazione di domande presenti all'interno del dataset TREC. Per valutare quanto ciascun modello impari dai dati di addestramento, i test sono stati ripetuti su sottoinsiemi di dimensione crescente: 0, 1, 10, 100, 1000 esempi e il dataset completo. Il caso *train_size = 0* è stato affrontato con approcci zero-shot specifici per ciascuna architettura.

1 Dataset

TREC è un dataset di domande in lingua inglese annotate con sei categorie semantiche le cosiddette *coarse labels*, nello specifico si tratta di: `ABBR` (abbreviazione), `ENTY` (entità), `DESC` (descrizione), `HUM` (persona), `LOC` (luogo), `NUM` (quantità). Le dimensioni originali prevedono 5452 esempi di training e 500 di test; il test set, in quanto tale non è mai stato utilizzato durante il training o la validazione.
Il sampling dei sottoinsiemi è stato effettuato con **stratified split** (via `scikit-learn`), in modo che ciascun sottoinsieme rispettasse la distribuzione originale delle classi. Questo passaggio è particolarmente importante nelle configurazioni few-shot (come ad es. $N = 10$), dove un campionamento casuale potrebbe escludere intere classi minoritarie impedendo di conseguenza ai modelli di conoscere la distribuzione di tali classi.

2 Fine-tuning di BERT

Come modello BERT è stato scelto `bert-base-uncased` da HuggingFace, con l'aggiunta di un layer lineare di classificazione a 6 output (`BertForSequenceClassification`). Il training è stato regolato tramite Cross-Entropy loss con ottimizzatore AdamW ($lr = 2e\text{-}5$), seguendo le indicazioni del paper originale di BERT per il fine-tuning.
La strategia di addestramento dipende dalla dimensione del training set:

- **train_size $<$ 50:** epoche fisse (`MAX_EPOCHS = 10`), senza validation split, perché con pochissimi esempi uno split del 20% ridurrebbe il training a dati insufficienti.
- **train_size $\geq$ 50:** early stopping su un validation set del 20% (patience = 3). I pesi migliori vengono salvati con `copy.deepcopy` e ripristinati al termine.

3 Supervised Fine-Tuning di GPT-2

GPT-2 è un modello autoregressivo, ovvero non classifica direttamente, ma genera testo token per token. Per adattarlo al task di classificazione abbiamo usato un approccio di **Supervised Fine-Tuning (SFT)** con il seguente prompt:

```
Classify the following question into one of these categories: ABBR,
ENTY, DESC, HUM, LOC, NUM.
Question: {testo}
Category: {label}
```

Durante il training, il modello deve imparare a generare solo la label corretta, non il testo del prompt. Per evitare che il gradiente venga calcolato anche sui token dell'istruzione, i token del prompt ricevono target label `-100`: PyTorch ignora nativamente questo valore nel calcolo della Cross-Entropy. In questo modo la loss viene calcolata esclusivamente sui token della categoria target.
La stessa strategia di early stopping descritta per BERT è stata applicata anche a GPT-2, con gli stessi criteri di soglia sulla dimensione del training set.

3.1 Prompt Engineering e Loss Masking

Il modello è stato istruito tramite un prompt strutturato che definisce il task e le categorie ammissibili:

> *"Classify the following question into one of these categories: ABBR, ENTY, DESC, HUM, LOC, NUM. Question: {testo} Category: {label}"*

Dal punto di vista implementativo, per evitare che il modello "impari a memoria" le istruzioni del prompt invece di concentrarsi sulla classificazione, è stata applicata la tecnica del **Loss Masking**. Nel `SFTDataset`, i token corrispondenti all'istruzione e alla domanda sono stati marcati con il valore target -100 . Poiché la funzione di *Cross-Entropy* in PyTorch ignora nativamente questo valore, il calcolo del gradiente e l'aggiornamento dei pesi avvengono esclusivamente sui token della categoria target.

3.2 Strategia di Training e Early Stopping

Come anticipato precedentemente, l'addestramento è stato modulato in base alla disponibilità dei dati per ottimizzare la convergenza:

- **Small Data ($N < 50$):** Il modello esegue un numero fisso di epoche (`MAX_EPOCHS` = 10), poiché un set di validazione ridurrebbe eccessivamente i dati di training già scarsi.
- **Large Data ($N \geq 50$):** Viene attivato l'**Early Stopping** con una *patience* di 3 epoche, monitorando la loss su un set di validazione del 20%. Questo garantisce il salvataggio della migliore istanza del modello tramite `deepcopy`, prevenendo l'overfitting.

3.3 Logits Fallback

Per garantire che il modello produca sempre una delle 6 etichette valide (ABBR, ENTY, DESC, HUM, LOC, NUM), è stato implementato un sistema di inferenza a due livelli nel metodo `predict_with_logits_fallback`:

1. **Generazione e Mapping:** Il modello genera liberamente dei token che vengono poi processati tramite *keyword matching* e *prefix matching*.

2. **Logits Fallback:** Qualora la generazione non sia riconducibile a nessuna classe, il sistema analizza i **logits** del token successivo al prompt. Vengono confrontate le probabilità grezze dei soli 6 token corrispondenti alle etichette TREC, selezionando l'indice con il valore massimo (*argmax*).

4 Classificazione Zero-Shot (train_size = 0)

Il caso zero-shot richiede soluzioni diverse per le due architetture, che non sono confrontabili direttamente. Questo riflette la loro natura fondamentalmente diversa.

BERT (NLI). Un encoder BERT senza fine-tuning non può essere usato direttamente per il task di classificazione poiché il layer lineare finale è inizializzato casualmente e produrebbe solamente predizioni casuali. La soluzione adottata è la **Natural Language Inference (NLI)**: utilizzando il modello `facebook/bart-large-mnli`, per ogni domanda di test il sistema calcola la probabilità che la domanda implichi ciascuna delle sei descrizioni testuali delle categorie (es. "This is a location question"). La categoria con la probabilità più alta diventa la predizione.

GPT-2 (prompt-based). GPT-2 nella versione base viene interrogato con il prompt sopra descritto, senza la label finale. Il testo generato viene mappato alla label corrispondente con tre livelli di fallback: (1) match esatto, (2) match per prefisso, (3) keyword matching. Se nessuno di questi riesce, il sistema confronta direttamente i logit del token successivo a `Category:` per ciascuna delle sei label valide, scegliendo la più probabile. Questo garantisce sempre una predizione definita.

5 Framework di valutazione

Al termine di ogni run, le predizioni vengono salvate in un file CSV con il formato:

```
text | true_label | true_label_name | predicted_label |
predicted_label_name
```

Lo script `evaluate.py` legge automaticamente tutti i CSV presenti nella cartella `results/` e calcola, per ciascun modello e dimensione di training, quattro metriche: Accuracy, F1-Micro, F1-Macro e F1-Weighted. Viene anche generata una figura con le curve comparative dei due modelli al variare di N (Figura 1). Il vantaggio di separare la generazione delle predizioni dal calcolo delle metriche è che i run computazionalmente costosi (in particolare per $N = 1000$ e $N = \text{Full}$) vengono eseguiti una sola volta e i risultati vengono conservati.

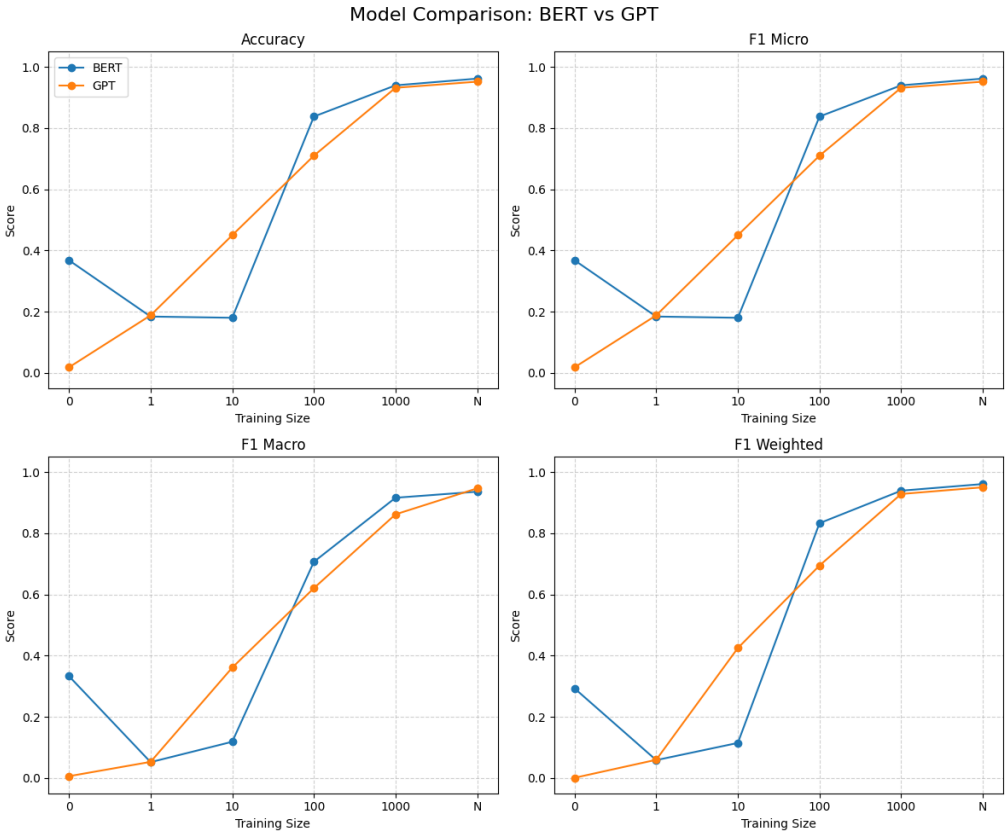

Figura 1: Confronto delle performance tra BERT e GPT-2 al variare di N.

6 Analisi dei risultati e Discussione

Le performance dei modelli BERT e GPT-2 SFT sono state valutate su sei configurazioni di training set (0, 1, 10, 100, 1000, Full), utilizzando l'Accuracy e il punteggio F1 (Macro e Weighted) come metriche di riferimento. Sulla base dei dati riportati nella Tabella 1, è

Tabella 1: Confronto analitico delle performance tra BERT e GPT-2 SFT al variare del numero di esempi di training.

Modello	**Metrica**	**0**	**1**	**10**	**100**	**1000**	**Full**
BERT	Accuracy	**0.368**	0.184	0.180	**0.838**	**0.940**	**0.962**
GPT-2 SFT	Accuracy	0.018	**0.188**	**0.450**	0.710	0.932	0.952
BERT	F1 Micro	**0.368**	0.184	0.180	**0.838**	**0.940**	**0.962**
GPT-2 SFT	F1 Micro	0.018	**0.188**	**0.450**	0.710	0.932	0.952
BERT	F1 Macro	**0.333**	0.052	0.119	**0.707**	**0.916**	0.936
GPT-2 SFT	F1 Macro	0.006	**0.053**	**0.361**	0.621	0.862	**0.947**
BERT	F1 Weighted	**0.293**	0.058	0.115	**0.833**	**0.939**	**0.961**
GPT-2 SFT	F1 Weighted	0.001	**0.060**	**0.425**	0.695	0.929	0.951

possibile trarre le seguenti conclusioni analitiche:

- **F1 Micro vs Accuracy:** Come si evince dalla tabella, i valori di *F1 Micro* e *Accuracy* sono identici per entrambi i modelli. Questo è teoricamente corretto per task di classificazione multiclasse *single-label*, in quanto il calcolo dei *Micro-Average* aggrega i contributi di tutte le classi portando allo stesso risultato dell'accuratezza globale.

- **Prestazioni Few-Shot ($N = 10$):** GPT-2 mostra un vantaggio netto e statisticamente rilevante con 10 esempi (Accuracy 0.450 vs 0.180). Questo conferma che l'approccio generativo con prompt è estremamente efficace quando la supervisione è minima, permettendo al modello di sfruttare meglio le proprie conoscenze.

- **Adattabilità di BERT:** Nonostante le difficoltà iniziali nei casi con pochissimi dati, BERT dimostra la sua robustezza non appena la dimensione del training set raggiunge $N = 100$, superando GPT-2 e mantenendo il primato fino alla configurazione Full (96.2%). Questo suggerisce che l'architettura *encoder* rimane superiore nel catturare la semantica necessaria alla classificazione quando è disponibile un volume di dati sufficiente.

6.1 Caso Zero-Shot ($N = 0$)

In assenza di dati di addestramento, si osserva una netta superiorità di BERT (36.8% di Accuracy) rispetto a GPT-2 (1.8%). Questa disparità risiede nella metodologia adottata:

- **BERT (NLI):** Utilizza un modello pre-addestrato per l'*Inference* (`facebook/bart-large-mnli`), che permette di classificare confrontando semanticamente la domanda con le etichette testuali.

- **GPT-2:** Essendo un generatore di testo puro, senza fine-tuning non possiede una comprensione intrinseca delle classi TREC; il sistema di *fallback* sui logit produce predizioni quasi casuali.

6.2 Caso Few-Shot ($1 \leq N \leq 100$)

Il dato più significativo emerge nella configurazione a 10 esempi, dove GPT-2 domina con un'Accuracy del 45% contro il 18% di BERT. Questa capacità di apprendimento rapido è dovuta a due fattori:

1. **Instruction Tuning:** Il prompt esplicita le categorie, permettendo a GPT-2 di "agganciare" la sua vasta conoscenza linguistica pre-addestrata al task specifico con pochissimi campioni.

2. **Allineamento Semantico:** Mentre BERT deve addestrare da zero una *classification head* lineare, GPT-2 deve solo imparare a completare la frase `Category:` con il token corretto, un'operazione coerente con il suo obiettivo di pre-addestramento.

6.3 Caso "High"-Data ($N \geq 1000$)

Al crescere del dataset, BERT recupera lo svantaggio iniziale e supera leggermente GPT-2, raggiungendo il picco di **96.2%** sul dataset completo. La natura bidirezionale dell'encoder BERT gli consente di catturare meglio le sfumature semantiche globali della

domanda, vantaggio che si manifesta quando la supervisione è abbondante. GPT-2 mantiene comunque performance eccellenti ($> 95\%$), confermando la validità dell'approccio SFT anche su larga scala.

7 Conclusioni

Il progetto mostra una pipeline di classificazione completa e riproducibile, che permette di confrontare direttamente BERT e GPT-2 in diversi set di addestramento. I due modelli adottano approcci zero-shot diversi basati sulla loro architettura nativa (NLI per l'encoder, prompt-based per il decoder), il che rende il confronto metodologicamente coerente con la letteratura recente. I risultati numerici sono consultabili tramite il framework di valutazione incluso nel repository.
L'esperimento dimostra che non esiste un modello universalmente superiore, ma la scelta dipende strettamente dal set di dati disponibile. Dunque, **BERT (NLI)** è la scelta migliore per scenari **Zero-Shot**. **GPT-2 SFT** risulta sensibilmente più efficace in contesti **Few-Shot** ($N = 10, 100$) grazie alla guida semantica del prompt. Tuttavia, entrambe le architetture convergono verso risultati di alta precisione con dataset completi, con un leggero margine superiore dato da **BERT** dovuto alla sua architettura encoder-only.